INVITED TALK

Cooperative AI-Assisted FormalVerification: Integrating LLMs with ModelChecking for Scalable Software Assurance

Prof. Lucas C. Cordeiro
Professor of Computer Science

The University of Manchester

UFAM

PPGEE
Programa de Pós-Graduação em Engenharia Elétrica - UFAM

Guess the number

310,531 C programs · 9 LLMs · every one model-checked

What fraction of the best model's code passes formal verification?

Model	Programs	VS %	VF %	VU %
CodeLlama-13B	11,838	**15.48**	52.71	29.52
Llama2-13B	17,779	12.36	51.30	31.78
Gemma-7B	36,787	11.62	67.01	16.30
Falcon2-11B	11,946	10.28	63.16	24.56
Gemini Pro 1.0	38,695	9.49	63.91	24.13
Mistral-7B	9,934	8.36	62.08	25.88
GPT-3.5-turbo	76,168	7.29	65.07	26.09
Falcon-180B	68,463	6.48	62.07	28.67
GPT-4o-mini	38,921	**4.23**	57.14	36.77
All nine	**310,531**	**8.27**	**62.07**	**26.99**

≈ 1 in 6

for the best model (15.48%); all nine: 1 in 12 (8.27%)

A failed check ≠ a broken program

It means we cannot certify it, and that is already the problem.

Not a model-release problem

No model passes even 1 in 6 times; the newest in the study (GPT-4o-mini) scored lowest.

Fluent is not correct: most AI-generated code cannot be certified today.

Tihanyi, Bisztray, Ferrag, Jain & Cordeiro, "How secure is AI-generated code: a large-scale comparison of large language models", Empirical Software Engineering 30(2):47, 2025, Table 8. VS = verified (k-induction, 500 s), VF = violation found, VU = unknown (timeout); ER = error (1.5–5%) omitted, so rows sum to < 100%.

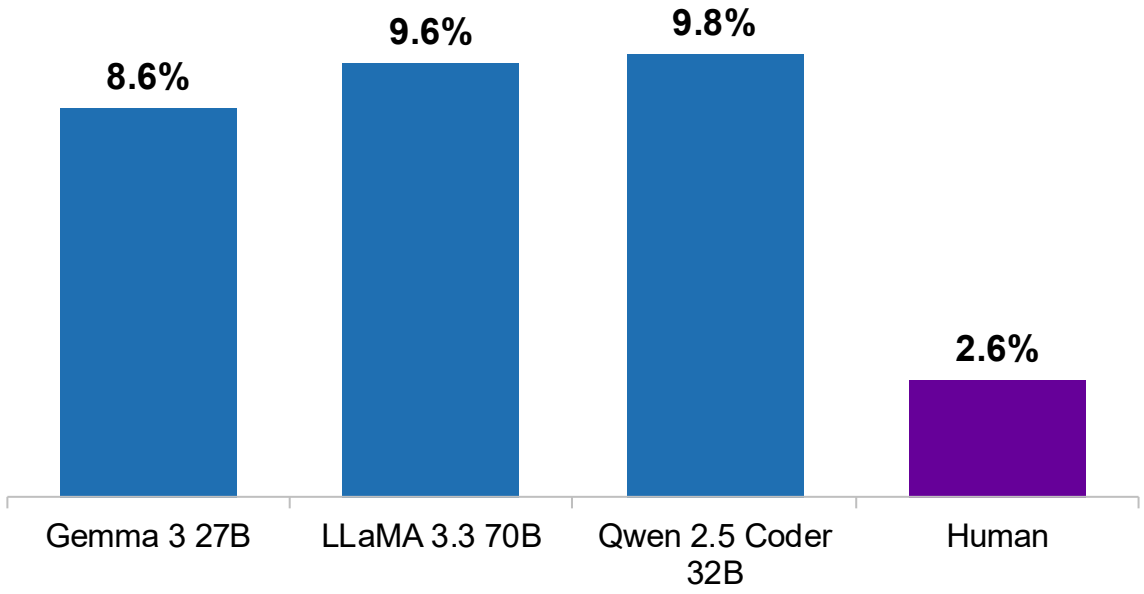
An independent check: AI-written vs human-written C++

UFAM

The University of Manchester

MANCHESTER 1824

Confirmed runtime violations (ASan/UBSan), by author



8,918 programs (7,223 by 3 LLMs, 1,695 by humans) · 851 contest tasks

Adjusted odds ratios (Table VI), 95% CI: runtime 2.15 [1.52, 3.03]; static (cppcheck) 0.68 [0.57, 0.81].

3.6×
raw · ≈ 2× odds, adjusted

AI vs human: confirmed runtime violations
9.3% vs 2.6% of compiled programs. Same task, similar length and test results: about 2× the odds remains.

No gap

under static analysis alone
cppcheck flags both alike; once length is accounted for, AI code even looks safer.

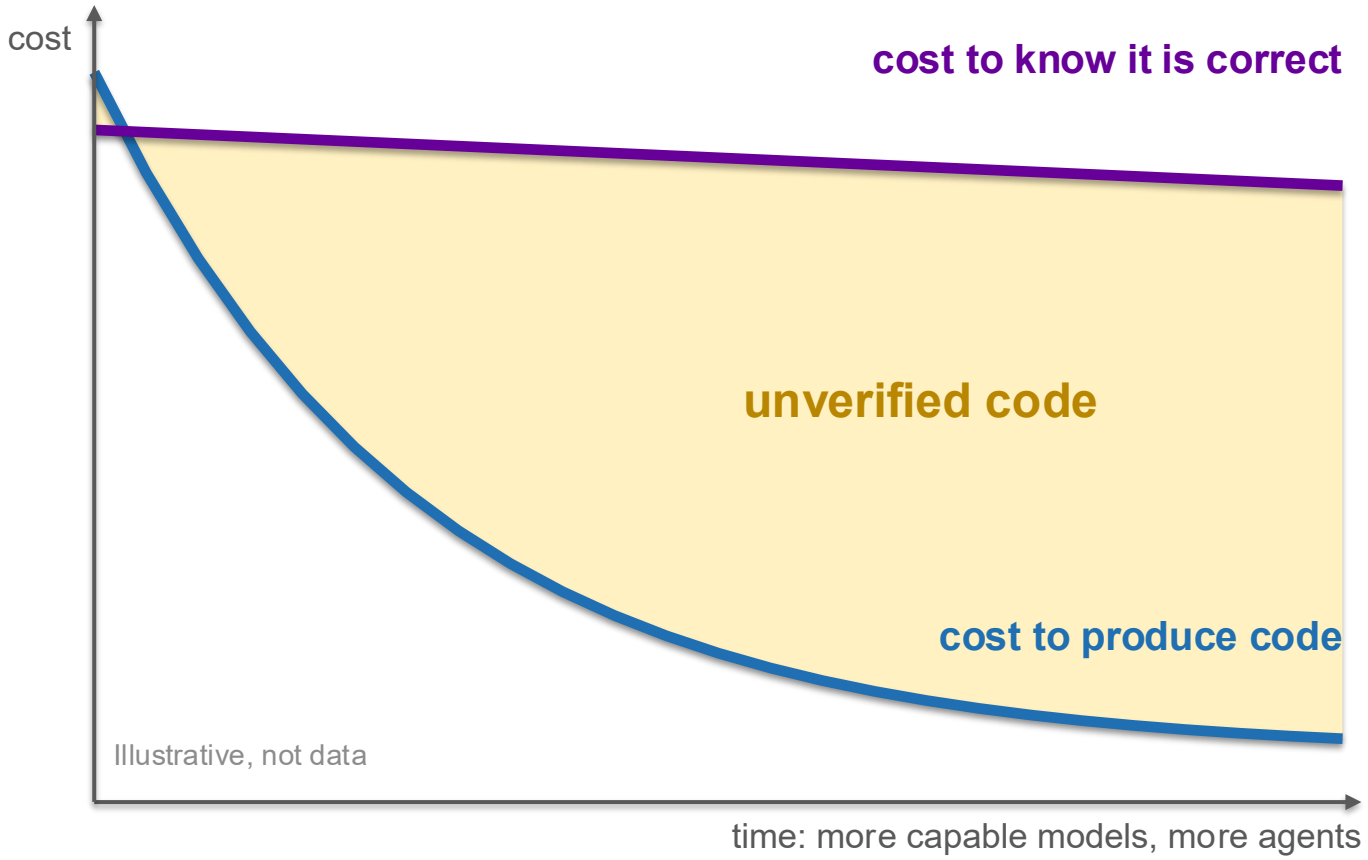
598 of 661

programs with an ESBMC-proven violation no sanitizer run flagged
Each has a concrete counterexample, found by exploring every input up to the bound.

> Static warnings say the code looks fine; runtime checks and proofs say otherwise, and catch different bugs.

Mahmud, Sikder, Ji, Zhang & Lei, “The Illusion of Safety: Multi-Tier Verification of AI vs. Human C++ Code”, arXiv:2607.00107, 2026 (preprint).

Code is getting cheap; knowing it is right is not



Generation keeps getting cheaper

Every model release lowers the cost of a line of code.

Checking does not, on its own

Review, tests and proofs scale with the code, not with the model.

Agents widen the gap

They produce more, and a human reads less of it.

> Better models lower the error rate. They do not remove the need for evidence.

From assisted to agentic: the reviewer disappears

Human-driven

Manual debugging

Root-cause analysis

Human writes, human checks

→

AI-assisted

Code suggestions

Test generation

Human still reviews each step

→

Agentic

Plans multi-step solutions

Acts using tools

Evaluates and iterates alone

?

Google: 75% of all new code is AI-generated, up from 25% in late 2024. Anthropic: over 80% of merged code is written by Claude. Microsoft: 20–30% of the code in its repositories. Microsoft's CTO predicts 95% by 2030.

Sources: Pichai, Cloud Next, Apr 2026 · Anthropic, "When AI builds itself", Jun 2026 · Nadella via TechCrunch, Apr 2025 · Scott via TechSpot

The question this talk exists to ask: who verifies that?

When an agent delivers a result instead of a step, no human reviews the steps, so a checker has to.

Why not more tests, or a second model?

UFAM

The University of Manchester

More tests

Tests sample inputs: in one study, 598 of 661 ESBMC-proven violations escaped every sanitizer run.*

```
int x = read_int(); // user
if (x == 0x13371337) // 1/2³²
  buf[x] = 0;        // OOB
// no test proves there is no other
```

An LLM reviewer

Swaps one unsound judge for another, often trained on the same data as the generator.

"Looks correct to me."

Formal verification

A proof over every input, or a concrete counterexample. But it has needed specialists to write properties and invariants.

base $\mathrm{Init}(s_0) \wedge \mathrm{T}(s_0 \ldots s_k) \Rightarrow \mathrm{P}(s_0 \ldots s_k)$

step $\mathrm{P}(s_0 \ldots s_k) \wedge \mathrm{T}(s_0 \ldots s_{k+1}) \Rightarrow \mathrm{P}(s_{k+1})$

k-induction: bounded checks, unbounded proof

And Rust? Safe Rust rules out memory corruption, not logic errors: that write becomes a panic, and a wrong result still compiles.

That specialist effort is the bottleneck, and it is exactly what a model is good at guessing.

> Use the model to guess what a proof needs, never to judge the result.

* Mahmud, Sikder, Ji, Zhang & Lei, "The Illusion of Safety: Multi-Tier Verification of AI vs. Human C++ Code", arXiv:2607.00107, 2026 (preprint).

Cooperation, not replacement: the fast and the sound

UFAM

The University of Manchester

MANCHESTER 1824

Language models

- Fast, fluent, general
- Guess what humans find hard: loop invariants, contracts, specifications, test inputs
- **No guarantee whatsoever**

The interface

is the research problem

What is passed, in what form, and what happens when it is wrong

Proof engines

- Exhaustive within their bound
- Verdicts are machine-checkable; a counterexample is executable
- **Expensive; need a spec**

The design rule we keep arriving at

Let the model propose anything. Let nothing through that the proof engine has not confirmed. Then a wrong invariant or test costs time, never correctness; a wrong specification still needs a human to catch it.

> LLMs are fast but unsound. Verifiers are sound but slow. A careful interface gives you both.

ESBMC: one intermediate representation language

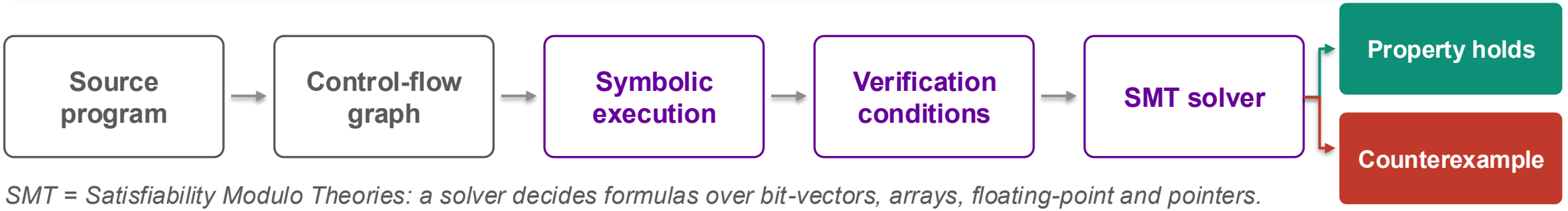
UFAM

The University of Manchester

MANCHESTER 1824

★ ASE 2023 Most Influential Paper Award (ASE 2009 paper)

ESBMC is an open-source, context-bounded model checker that **detects or proves the absence of runtime errors** in single- and multi-threaded C, C++, CUDA, CHERI, Java/Kotlin, Python, Rust and Solidity.



SMT = Satisfiability Modulo Theories: a solver decides formulas over bit-vectors, arrays, floating-point and pointers.

k-induction adds an inductive step, so a property can be proved for loops of any length.

Open source

Apache 2.0 (mainly): no licence negotiation to trial it.

Two useful answers

A proof, or an input that makes the program fail.

Funded by

Arm, EPSRC, Ethereum Foundation, Rust Foundation, EU H2020 ELEGANT, UKRI Soteria.

> The output is evidence a machine can re-check, not a confidence score.

How it works, in one example

```
def main() -> None:
    a = [nondet_int(),
         nondet_int()]
    i: int = nondet_int()
    x: int = nondet_int()
    if x == 0:
        a[i] = 0
    else:
        a[i + 2] = 1
    assert a[i + 1] == 1
```

nondet_int() makes i, x and a symbolic: no test suite covers every combination.

```
g1 = x1 == 0
a1 = a0 WITH [i0 := 0]
a2 = a0
a3 = a2 WITH [2+i0 := 1]
a4 = g1 ? a1 : a3
t1 = a4[1+i0] == 1
```

All paths become one logical formula; none is missed.

Ask the SMT solver:

$$C \wedge \neg P$$

satisfiable → a bug, with the exact i, x and array contents that trigger it

unsatisfiable → no such execution exists within the bound

C = the program's constraints. P = the property you want.

Unfold loops to a bound

Slicing, constant propagation, caching

Convert to single-assignment form

Extract constraints C and property P

Check satisfiability of C ∧ ¬P

BMC is complete only up to the bound; k-induction and loop invariants make the proof unbounded.

k-induction: from bounded search to unbounded proof

Base case B(k)

BMC to depth k: finds real bugs

Forward condition F(k)

All loops unrolled within k: a proof

Inductive step I(k)

Holds for k steps, so for the next: a proof

Four live runs

Bug	**found, k = 11**
Bounded loop	**F(k) proves, k = 21**
Unbounded loop	**UNKNOWN**
+ one invariant	**I(k) proves, k = 2**

SV-COMP ReachSafety runs --k-induction.

Plain k-induction stalls on the unbounded loop; one invariant closes the proof.

A counterexample is a test; a proof is a certificate

```c
// illustrative example, not production code
int copy_bytes(const unsigned char *in, size_t n,
               unsigned char out[4]) {
  if (n > 4) return -1;
  for (size_t i = 0; i <= n; i++)   /* off-by-one */
    out[i] = in[i];
```

Two kinds of witness

Violation → a concrete input, replayed as a regression test.

Correctness → a certificate an independent validator re-checks.

1 · ESBMC finds the bug

```
$ esbmc copy_bytes.c \
    --unwind 6 \
    --generate-ctest-testcase
  n = 4
  array bounds violated
VERIFICATION FAILED
```

2 · It becomes a C program

```
// test_case.c (generated)
…nondet_uint(void) {
  static unsigned int const
      v[] = { 4 };  …
$ ./test_case
ASan: stack-buffer-overflow
```

3 · Fix i < n: test passes, proof holds

```
$ ./test_case  →  exit 0

$ esbmc copy_bytes-fixed.c \
    --k-induction
VERIFICATION SUCCESSFUL
all states reachable, k = 6
```

> The bug report is an executable test that fails today and passes after the fix.

Every failing property becomes its own test

```
// illustrative example, not production code
unsigned apply(unsigned op, unsigned idx,
               unsigned val, unsigned out[4]) {
  if (op == SET) out[idx] = val;  /* bug 1 */
  if (op == DIV) return val / idx; /* bug 2 */
  return 0;
```

Why a CTest suite

One test per failure → one command runs them all, in CI.

One test per input → an input that hits two bugs gives one test.

1 · Check every property

```
$ esbmc apply.c \
    --multi-property \
    --generate-ctest-testcase
FAILED  array bounds
FAILED  division by zero
VERIFICATION FAILED
```

2 · Two tests, two failures

```
// test_case_1.c
   op = 0, idx = 4294967295
// test_case_2.c
   op = 1, idx = 0
$ ctest  →  0% passed
ASan SEGV · UBSan div by 0
```

3 · Fix both: suite passes, proof holds

```
$ ctest  →  100% passed

$ esbmc apply-fixed.c \
    --multi-property
VERIFICATION SUCCESSFUL
all 3 properties PASSED
```

One run checks every property; each failure is a test you keep after the fix.

Three places a guess can enter

The proposer need not be an LLM: a fuzzer is just as fast and just as unsound.

A FuSeBMC

Test inputs and seeds
fuzzers and BMC propose

checked by
Execution: a goal is covered, or it is not

B ibmc

Loop invariants
an LLM proposes

checked by
Vampire checks it is inductive; ESBMC uses it in the proof

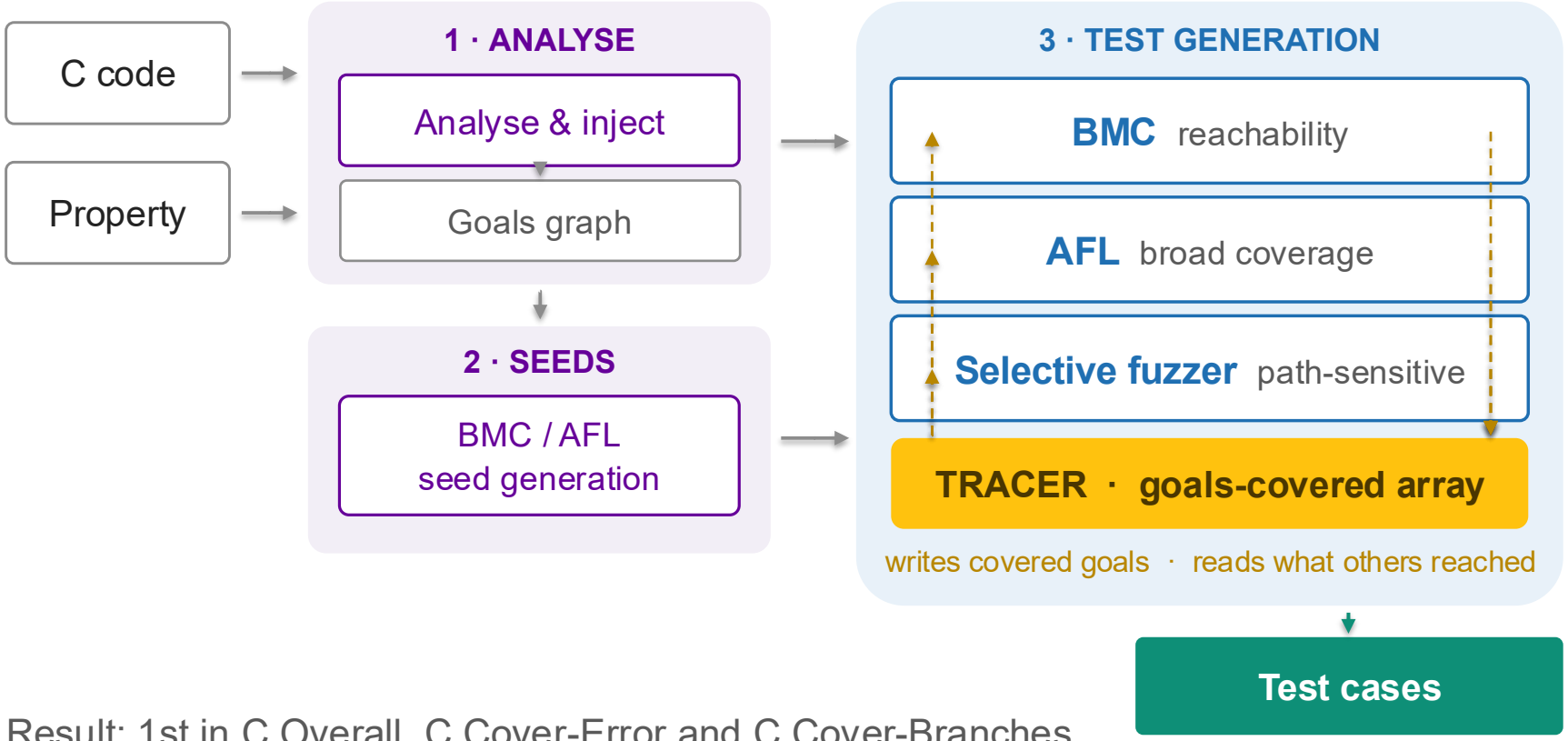
C SpecVerify

The property itself, from prose
an LLM proposes

checked by
ESBMC checks the code; only a human can confirm the property means what was intended

The proposal is cheap and the check is never optional; in C, the check also needs a human.

FuSeBMC: a portfolio with a shared coordinator



Three engines

Each strong where the others are weak.

One coordinator

The Tracer records covered goals in shared memory.

It feeds back

New coverage becomes a seed for the next round.

Result: 1st in C.Overall, C.Cover-Error and C.Cover-Branches at Test-Comp 2026.

No engine talks to another: they cooperate only through a shared record of covered goals.

Judged by blind competition, not by press release

Test-Comp 2026

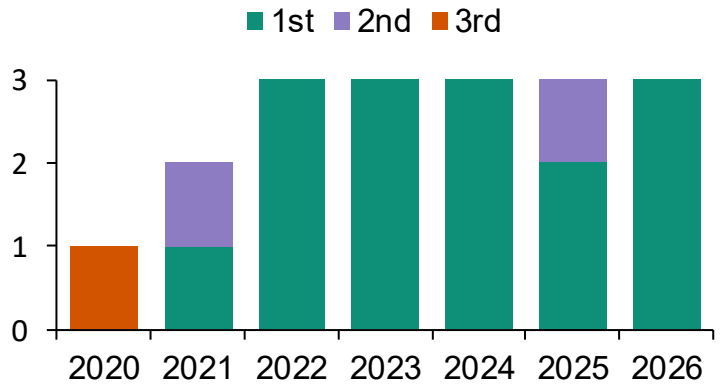
FuSeBMC: 1st in C.Overall, C.Cover-Error and C.Cover-Branches.

SV-COMP 2026

ESBMC-kind: 2nd in C.ReachSafety (gold: CPAchecker).

Rust Foundation, Aug 2025

ESBMC approved for the Rust standard-library verification effort, with Flux, Kani and VeriFast.



Medals per year: ESBMC 2020, FuSeBMC 2021–26

The same discipline, every year

- Benchmarks change yearly, so scores aren't comparable year to year.
- The method is constant: public tasks, a blind run, organisers who are not us.
- Hold AI testing tools to the same standard: a shared benchmark, a blind run, a published table.

Annual, blind, third-party benchmarking, the same evidence any competitor is judged on.

test-comp / sv-comp.sosy-lab.org/2026 · rustfoundation.org, 7 Aug 2025 · Disclosure: FuSeBMC is maintained by VeriBee, where I am CSO.

The bound problem: 100 unrollings for one assertion

```
x = 0; y = 50
# copy 1
x = x + 1
if x > 50: y = y + 1
# copy 2
x = x + 1
if x > 50: y = y + 1
# … 98 more copies
assert y == 100
```

Correct, but only after 100 unrollings.
(Illustrative; ibmc's benchmarks are C.)

An inductive invariant replaces the loop

Holds on entry, preserved by every iteration:

```
__ESBMC_loop_invariant(0 <= x and x <= 100)
__ESBMC_loop_invariant(not (x <= 50) or y == 50)
__ESBMC_loop_invariant(not (x > 50) or y == x)
```

The bound disappears.

A wrong invariant costs time, never correctness.

> Invariant synthesis is the classic bottleneck in automated verification, and a good fit for LLMs.

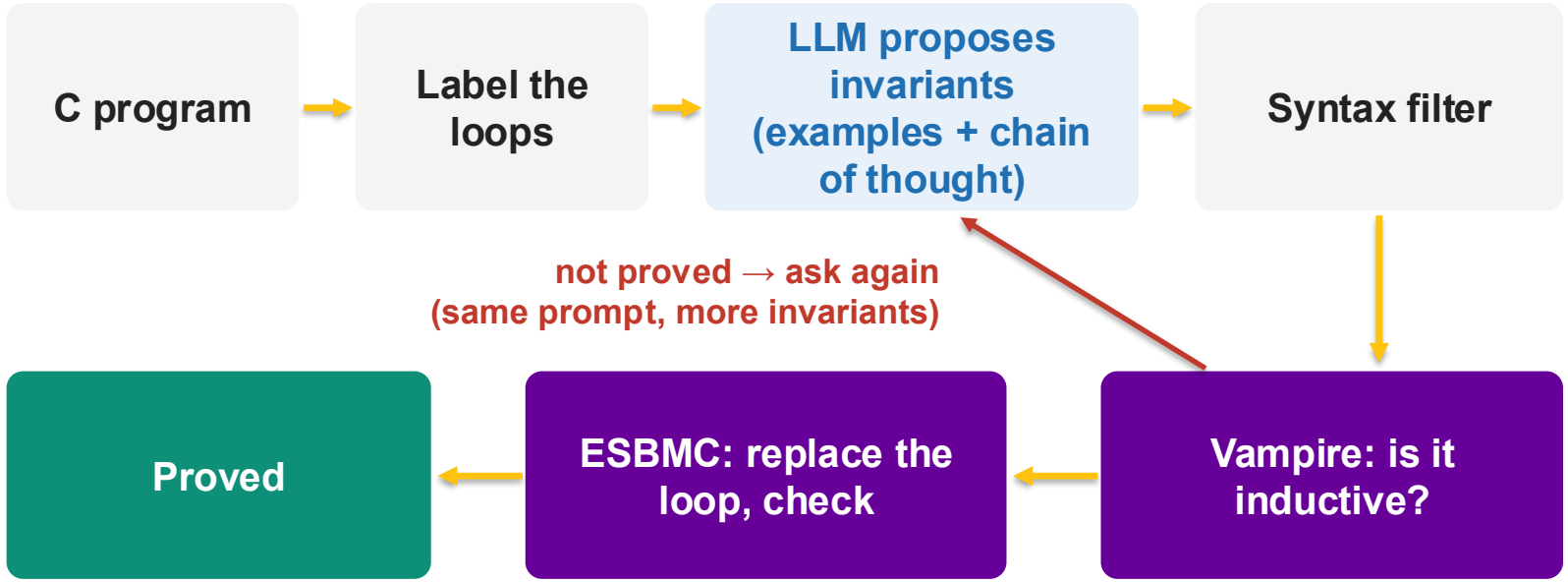
ibmc: the LLM proposes, two proof engines dispose

UFAM

The University of Manchester

MANCHESTER 1824

★ ACM SIGSOFT Distinguished Paper Award, ASE 2024



Nothing the LLM proposes reaches the proof until Vampire has shown it is inductive.

Sound, not complete

The LLM's output is never trusted, only checked: it can cost time, never add a wrong proof.

Constrain the output

67–75% of tries don't parse.

Guide the format, then filter

Filtering alone limits flexibility.

Balance the prompt

Too much explanation distracts.

Soundness is preserved by construction: nothing the model says is trusted, only checked.

Pirzada, Reger, Bhayat & Cordeiro, "LLM-Generated Invariants for Bounded Model Checking Without Loop Unrolling", ASE 2024, pp. 1395–1407. DOI 10.1145/3691620.3695512.

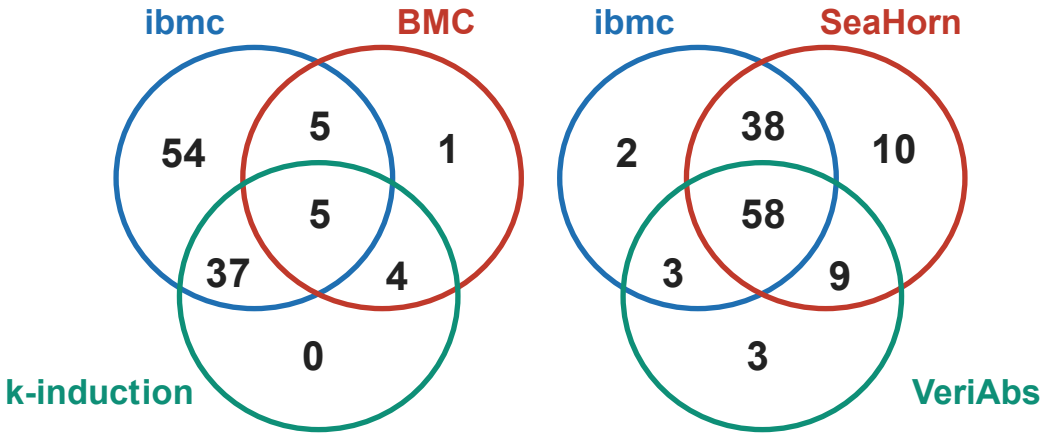
Up to 101 of 133 programs proved, none of them wrongly

Benchmark programs solved

16	46	101	115
plain BMC	k-induction	ibmc*	SeaHorn

*Union of six prompt configurations; any single one solves ~58% (≈77 of 133), still well above k-induction (46). We do not win outright: SeaHorn solves 115.

Which programs each engine reaches



Sound by construction

None of the 101 contradicts the benchmark's ground truth. The LLM cannot make a verdict wrong, only slower.

Complementary, not redundant

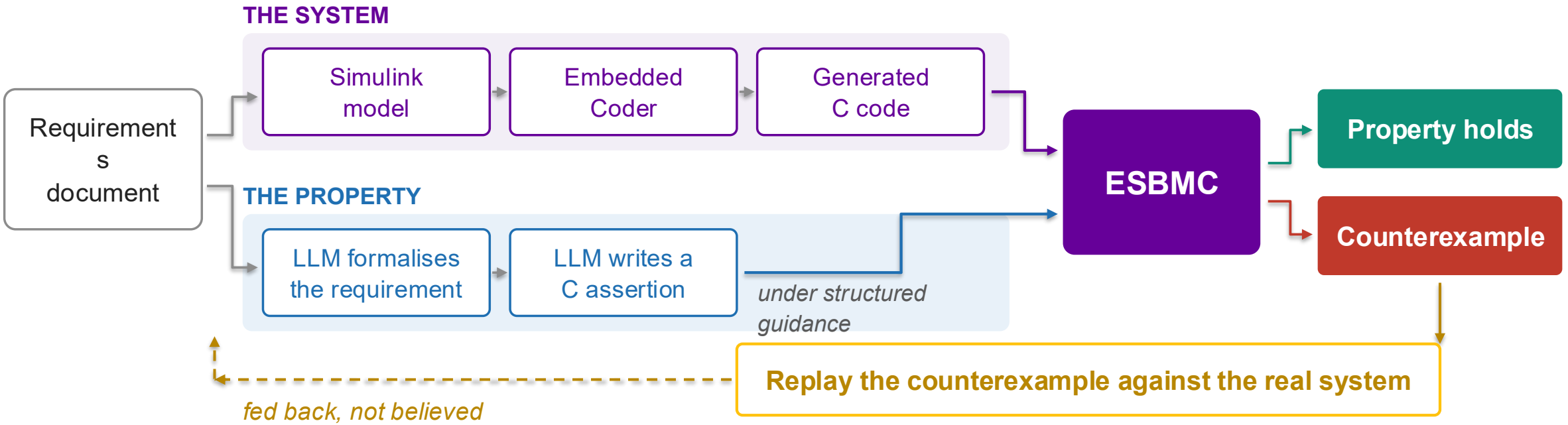
Different engines miss different programs: the argument for a portfolio.

Each set of invariants proves programs the other misses: LEMUR's 6, ours 10 (backup C1).

> The interesting result is not the total; it is that different engines miss different programs.

Pirzada, Reger, Bhayat & Cordeiro, ASE 2024, Table 3 (code2inv, 133 programs; ibmc = union of 6 configurations)

SpecVerify: from prose to a checked property



Industrial benchmarks

Lockheed Martin CPS challenge problems, Simulink to C.

Two phases

Formalise the requirement, then turn it into a C assertion.

The loop that closes it

The LLM is never the final authority; a property that fails is fed back.

> The specification is the bottleneck in industrial verification, and it is written in prose.

Wang, Farrell, Cordeiro & Zhao, “Supporting Software Formal Verification with Large Language Models: An Experimental Study”, IEEE RE 2025, pp. 423–431, doi:10.1109/RE63999.2025.00049.

“The value should be reasonable”

Requirement
“The value should be reasonable.”

→ **LLM** →

```
assert(bottom < x && x < top);
```

Which bounds? “Reasonable” has no semantics.

```
double range1, limit1, limit2, …;
```

Which limit did the requirement mean?

Full automation stops here: someone must decide the limit.

The model

Picks one silently, with full confidence. Nothing tells you it guessed.

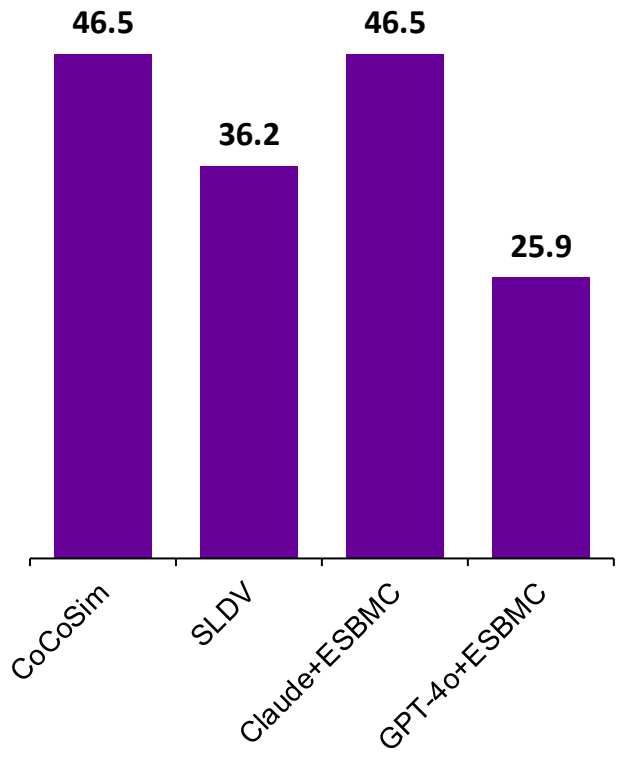
The checker

A too-strict pick shows up as a false alarm. A too-loose one passes silently: the checker cannot see it.

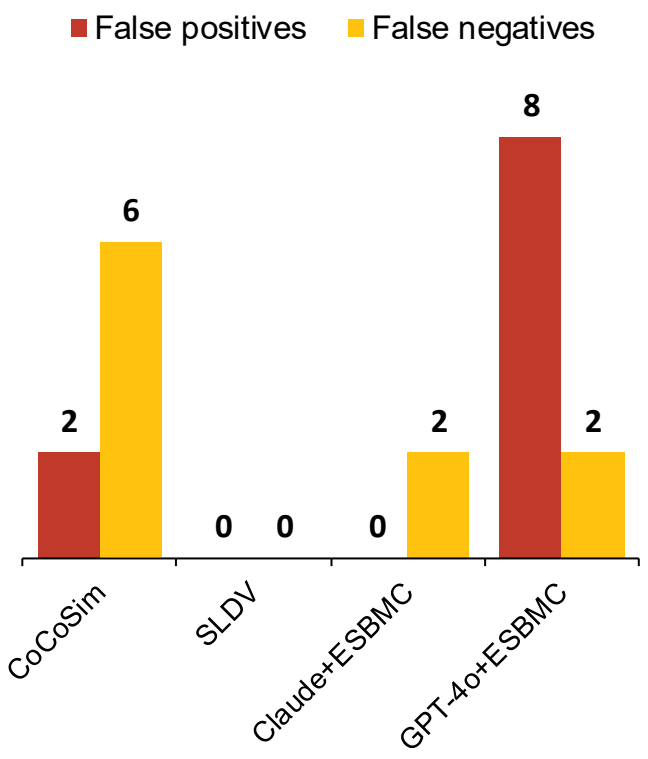
Here a wrong guess can cost correctness, not just time: the specification needs a human.

Expert-level agreement, with no false positives

Verification rate (%)



False positives / false negatives



79.31% match expert-written properties (46 of 58)

0 false positives: the verifier never cried wolf

2 previously unreported floating-point errors found

Read the chart carefully

The verification rate, 46.5% (27 of 58), equals CoCoSim's. SLDV also has no false alarms but verifies fewer (36.2%).

Claude 3.5 Sonnet + ESBMC still missed 2 violations: a wrong specification costs correctness.

> The model choice matters: with GPT-4o, the same pipeline produced 8 false positives.

Wang, Farrell, Cordeiro & Zhao, IEEE RE 2025, pp. 423–431. SLDV = Simulink Design Verifier; CoCoSim = NASA's Simulink verification framework.

Three systems, three cooperation interfaces

System	What cooperates	The interface	What it bought us
FuSeBMC	Model checker, coverage fuzzer, selective fuzzer	A shared-memory array of covered goals	1st in all three C categories, Test-Comp 2026
ibmc	LLM proposes invariants; Vampire and ESBMC check them	Prompt → candidate invariant → inductiveness check → proof	Up to 101 of 133 proved, vs 46 for k-induction. Sound by construction
SpecVerify	LLM formalises requirements; ESBMC checks the generated C	Prose → assertion → verdict → counterexample replayed against the system	79% agreement with experts, 0 false positives, 2 missed; a human must vet the spec

The generative side may be wrong; the checking side may never be skipped.

Cooperation gives you speed and correctness, if the interface is designed on purpose.

Production code, not benchmarks

8
codebases, every harness public

2
CVEs in the AWS SDK for C++

18
fixes merged upstream

Codebase	Findings	Fixed upstream
AWS SDK for C++	12	7, incl. 2 CVEs
NVIDIA OpenSMA firmware	11	1 reported: confirmed, fixed
vLLM	8	5
Google Chromium Dashboard	10	3
AWS Neuron NKI samples	2	2
boto3 · Related Website Sets · PyTorch	6	0 (1 filed)

An LLM agent drafted the harnesses; ESBMC, seeded-bug twins and maintainers confirmed them.

Counts per repository: the eight campaign repositories under github.com/lucasccordeiro

…and what we proved

Codebase	Proved to hold	Scope	Proofs
NVIDIA OpenSMA	Firmware contracts, by k-induction	unbounded	22
Google Chromium Dashboard	Approval integrity, vote authorization	bounded inputs	22
AWS SDK for C++	DateTime fix overflow-free; UUID memory-safe	bounded inputs	22
vLLM	KV-cache invariants, rounding contracts	bounded inputs	21
AWS Neuron NKI samples	Shape and bounds safety of 19 kernels	mixed	19
boto3 · RWS · PyTorch	Fix models; fused ≡ unfused QKV projection	bounded inputs	15

Checked for vacuity

Proofs are paired with seeded-bug controls that must FAIL; zero-VCC runs are rejected.

Why mostly bounded?

Loops are fully unwound, a complete proof for the loop, but input sizes are capped (e.g. ≤ 1,000 votes, ≤ 33-character UUIDs).

> The harness that exposes the bug also proves the fix, within stated bounds.

AWS SDK for C++: two CVEs in one Base64 decoder

CVE-2026-19642 · heap write

`"AAAA="`

The size counts '='; the loop does not. 2-byte buffer, write to buffer[2].

CVE-2026-19643 · read (signed char)

`"\xFF\xFF\xFF\xFF"`

Signed char 0xFF becomes a huge index into a 256-entry table.

1.11.862
fixes both CVEs

+2
DateTime overflows fixed

+3
cJSON defects fixed

AWS bulletin 2026-080-AWS · GHSA-wxx3-prfc-69xx, GHSA-mxm9-xpf9-x66x · PRs #3882, #3896, #3921 · github.com/lucasccordeiro/aws-sdk-cpp

> The counterexample was the bug report: a concrete input, not a warning.

NVIDIA OpenSMA: one bad index aborts the firmware

SetEpId request → validate() → on_set_endpoint_id() → set_cur_eid()

```
routing_table.ec.cur_eid.at(interface) = eid;   // cur_eid has 2 entries
```

Counterexample: iface = 2

Any 2–17 passes the validator; at() fails its bounds check and, built with -fno-exceptions, aborts: denial of service, not memory corruption.

Confirmed and fixed by NVIDIA

“The bound check is now mirrored into set_cur_eid()” (issue #1, May 2026).

Our abstract says “out-of-bounds write … from one network packet”. The harness shows an abort, and the interface index comes from the transport, so one packet is not demonstrated.

> The getter had the bounds check; the setter did not. ESBMC found the asymmetry.

github.com/NVIDIA/OpenSMA/issues/1 · github.com/lucasccordeiro/NVIDIA-OpenSMA

Google Chromium Dashboard: the API trusts its inputs

`?start=5&end=3`	HTTP 500, not 400	**fixed**
`?start=0`	milestone 0 accepted, null dates	**fixed**
`{"state": 0}`	vote skips the validator, HTTP 500	**fixed**
`PATCH {}`	missing key, HTTP 500	**open**

48 verification targets

10 findings

3 fixed upstream

> Web APIs fail like CLIs do: an admissible input that nobody validated.

Harnesses model the handlers in ESBMC-Python; inputs replayed in CPython · GoogleChrome/chromium-dashboard PRs #6451, #6452, #6656, issue #6464

vLLM: verifying the engine that serves the models

A malformed flag passes every validator, then crashes, hangs or misbehaves.

29	8	5 of 6
verification targets	live findings, 6 upstream issues	issues fixed upstream

`--block-size 0`	ZeroDivisionError at engine init	**fixed**
`--hash-block-size -k`	infinite loop at init	**fixed**
`--max-logprobs <neg>`	silently accepted	**fixed**
`--num-gpu-blocks-override 0`	bare AssertionError at engine init	**open**

> The stack that serves the models is ordinary software, and it breaks in ordinary ways.

vllm-project/vllm issue #43496, PR #43794 · github.com/lucasccordeiro/vllm (29 targets in the manifest)

What a re-runnable harness looks like

```
def main() -> None:
    user_block_size = nondet_int()
    __ESBMC_assume(0 <= user_block_size)
    __ESBMC_assume(user_block_size <= INT_BOUND)
    max_model_len = nondet_int()    # likewise

    q = cdiv(max_model_len, user_block_size)
    assert q * user_block_size >= max_model_len
```

```
$ esbmc 8-vllm-block-size.py
  user_block_size = 0
VERIFICATION FAILED

$ esbmc …-fixed.py
VERIFICATION SUCCESSFUL
```

nondet any CLI value

assume what holds

assert the contract

Read the VCC count

Zero VCCs means nothing was checked.

A harness is a test that quantifies over every input, and anyone can re-run it.

Five lessons from doing this on real code

1. **Zero false positives beats a higher proof rate**
   Developers stop trusting a tool that raises false alarms, then stop running it (Coverity; Google Tricorder).

2. **Maintainers fix what arrives with a concrete input**
   18 fixes merged upstream across five codebases.

3. **The generator's quality still propagates**
   SpecVerify with GPT-4o instead of Claude 3.5 Sonnet: 8 false positives, not 0.

4. **We do not win every benchmark**
   SeaHorn proves more than ibmc; CoCoSim matches SpecVerify. Portfolios beat champions.

5. **Verifiers have traps too**
   Zero VCCs, disabled unwinding assertions, contracts that silently check nothing: the evidence needs checking.

> These are the results a paper talk leaves out.

Coverity: Bessey et al., CACM 53(2):66–75, 2010 · Tricorder: Sadowski et al., ICSE 2015, pp. 598–608

Three challenges we must solve together

1 Verification at generation speed

Agents generate code in seconds; industrial proofs take minutes to hours. Checking must reuse earlier proofs.

Industry brings
agent traces

Academia brings
incremental proof reuse

2 Guarantees for agent workflows

Verifying each agent is not enough to verify the workflow they form.

Industry brings
real workflows

Academia brings
compositional contracts

3 Evidence we can trust

DO-178C already accepts formal evidence (DO-333). Who checks the checker?

Industry brings
certification needs

Academia brings
witnesses, validators

Every one of these is a joint problem: neither community solves it alone.

ACKNOWLEDGEMENTS

The people and funders behind this work

Whose work you saw

FuSeBMC
Kaled Alshmrany · Mohannad Aldughaim · Ahmed Bhayat

ibmc
Muhammad A. A. Pirzada · Giles Reger · Ahmed Bhayat

SpecVerify
Weiqi Wang · Marie Farrell · Liping Zhao

FormAI
Norbert Tihanyi · Tamas Bisztray · Mohamed Amine Ferrag · Ridhi Jain

ESBMC
Tong Wu · Rafael Sá Menezes, and all ESBMC contributors

IN CLOSING

LLMs are fast but unsound.
Proof engines are sound but slow.
Cooperation gives you both,
if you design the interface on purpose.

UFAM

The evidence

Best of nine LLMs: 1 in 6 programs verifies (all: 1 in 12). ibmc: 46 → up to 101 of 133. 2 CVEs; 18 fixes merged upstream.

The rule

Let the model propose anything. Let nothing through that the proof engine has not confirmed.

The ask

Nominate one component. We point ESBMC at it and return a counterexample or a proof. Apache 2.0.

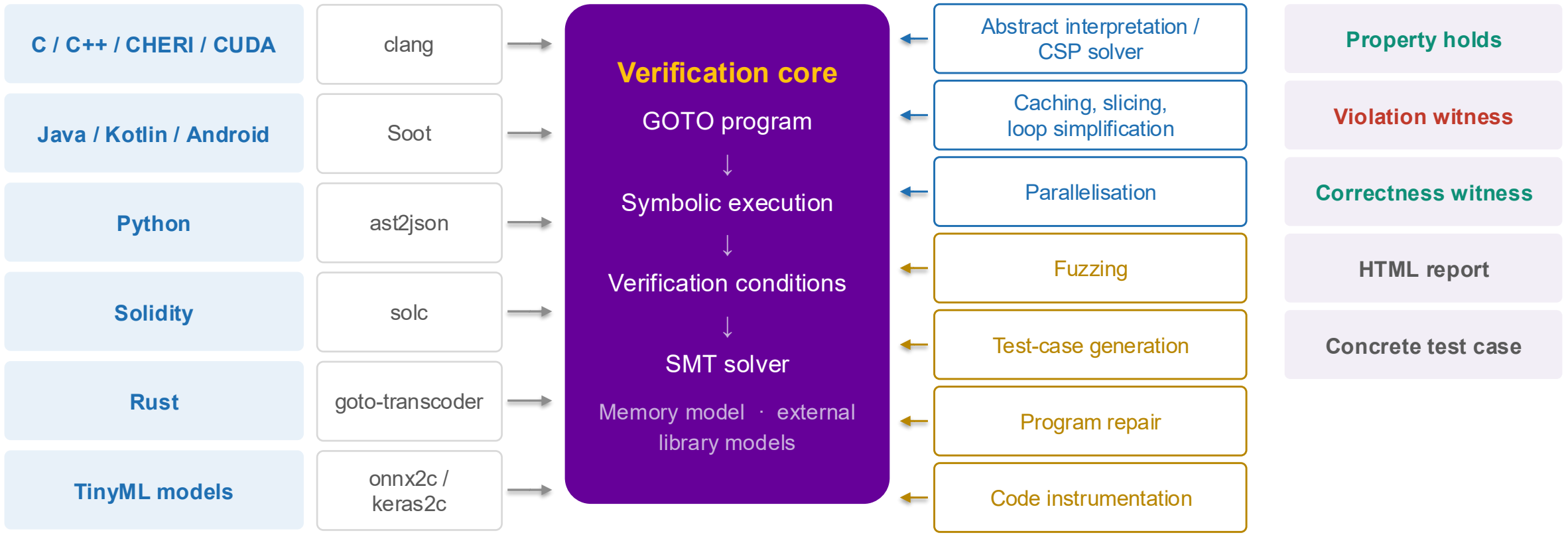
Prof. Lucas C. Cordeiro
lucas.cordeiro@manchester.ac.uk

cs.manchester.ac.uk/research/expertise/systems-and-software-security/ · github.com/esbmc/esbmc

MANCHESTER 1824
The University of Manchester

Thank you. Questions?

ESBMC in full: six front-ends, one verification core



Blue: components that cooperate with other analysis engines. Gold: components that consume or produce artefacts for other tools, the surface a new generative component attaches to.

> Every language reduces to one intermediate form, so a new front-end inherits the whole engine.

Turn "repeat until" into "repeat k times"

The worked example has no loop, so unfolding leaves it unchanged. Here is the step on a program that has one: the loop from Pattern B.

```
while x < 100:
    x = x + 1
    if x > 50:
        y = y + 1
```

The loop as written runs an unknown number of times.

```
if x < 100:
    x = x + 1
    if x > 50: y = y + 1
    if x < 100:
        x = x + 1
        if x > 50: y = y + 1
        assert x >= 100   # unwinding assertion
```

Unfolded to bound k = 2: two nested copies, plus a check that k was enough.

- If the unwinding assertion can fail, k was too small: ESBMC reports a violated unwinding assertion rather than call the program safe. --incremental-bmc and --k-induction raise k automatically.
- This loop needs k = 100 to be sure. Unfolding 100 copies is exactly the cost k-induction and LLM-generated invariants exist to avoid.

> Unfolding is cheap when k is small and expensive when it isn't.

Shrink the formula before the solver ever sees it

MANCHESTER 1824
The University of Manchester

Constant propagation

```
n = 4
r = n * 2
# becomes: r = 8
```

A value known at compile time replaces every place it is used.

Slicing

```
report = build_report()
assert x > 0
# report is dropped:
# it never reaches the property
```

Remove code that cannot affect the property being checked before solving.

Caching

```
p = f(x) + 1
q = f(x) - 1
# f(x) computed once,
# reused for q
```

An expression computed more than once is computed only once.

None of these fire on the ten-line worked example; it is already minimal. They matter at the size ESBMC actually verifies: thousands of lines, where the unoptimised formula would be too large for a solver to finish.

> Optimisation never changes what's true. It changes whether the solver finishes.

Every line becomes one equation, for both branches at once

MANCHESTER 1824
The University of Manchester

SSA = Static Single Assignment: every variable is written exactly once. Back to the worked example.

Python source	SSA form
a = [nondet_int(), nondet_int()]	a0 = a fresh, unconstrained array
i: int = nondet_int()	i0 = a fresh, unconstrained integer
x: int = nondet_int()	x1 = a fresh, unconstrained integer
if x == 0:	g1 = x1 == 0
a[i] = 0 (if-branch)	a1 = a0 WITH [i0 := 0]
else: a[i + 2] = 1 (else-branch)	a2 = a0; a3 = a2 WITH [2+i0 := 1]
(the two branches merge)	a4 = g1 ? a1 : a3
assert a[i + 1] == 1	t1 = a4[1+i0] == 1

a4 = g1 ? a1 : a3 is the whole trick: both branches live in one formula, so no path is discarded before the solver sees it.

> This SSA form is the same maths, whichever language the program started in.

What the program does, versus what you hoped was true

MANCHESTER 1824
The University of Manchester

C, the program's meaning

```
g1 = (x1 == 0)  ∧
a1 = a0 WITH [i0 := 0]  ∧
a2 = a0  ∧
a3 = a2 WITH [2+i0 := 1]  ∧
a4 = g1 ? a1 : a3
```

every SSA equality, conjoined

P, the property

```
t1
```

the assert's condition, and nothing else

Why check ¬P, not P?

- An SMT solver natively answers one question: is this formula satisfiable?
- To prove P always holds, ask the solver to satisfy its opposite, ¬P.
- If it can't, P holds everywhere; if it can, the satisfying assignment is a counterexample to P.

C is everything the program could do. P is the one thing you hoped was always true.

One satisfying assignment is all it takes

The solver returns satisfiable and hands back a concrete assignment. Trace it through the worked example:

`i = 0,  x = 0`	x == 0, so the if-branch runs
`a[i] = 0`	sets a[0] = 0, no effect on the property
`a[1] = 0  # solver's pick`	the property never told the program to set a[1]
`assert a[i + 1] == 1`	reads a[1] == 1 → asserts 0 == 1 → false

Why a passing test proves nothing here

- Nothing in the program constrains a[1], so whether a test fails depends on whatever a[1] happened to contain.
- The solver did not get lucky: it searched the SSA formula, proved an assignment exists that makes the property fail, and reported one.

This is the counterexample the main talk promised: concrete i, x, and array contents.

Competition record: what the numbers do and do not show

The University of Manchester

From the official results pages: 22 SV-COMP and 18 Test-Comp medals, 2012–2026.

Year	SV-COMP (ESBMC)	Test-Comp (FuSeBMC / ESBMC)
2026	2nd C.ReachSafety	1st C.Cover-Error · 1st C.Cover-Branches · 1st C.Overall
2025	2nd ReachSafety	1st Cover-Error · 2nd Cover-Branches · 1st Overall
2021–24	None	2021: 1st Cover-Error · 2nd Overall. 2022–24: 1st in all three, each year
2016–20	1st, 2nd (2016) · 2nd, 3rd (2017) · 3rd (2018, 2019, 2020)	2020: 3rd Cover-Error (ESBMC)
2012–15	5 × 1st · 2 × 2nd · 6 × 3rd, incl. Overall 2012, 2013	None (Test-Comp began in 2019)

Two things not to say

→ SV-COMP awards medals per meta category only: first places in sub-categories (ReachSafety-XCSP in 2021, MemSafety-Juliet in 2024) are not medals.

→ The 2026 gold in Java.Overall belongs to JBMC, not ESBMC; ESBMC-kind took one silver, in C.ReachSafety. Affiliations differ; see the speaker notes.

> Only the itemised 40 is checkable line by line; that is the number to quote.

Roadmap: automated formal verification of requirements

The University of Manchester
MANCHESTER 1824

1 Formalization — Completed

Automated requirements formalization (SpecVerify)

2 Decomposition — In Progress

System-level property decomposition with refinement

3 Auditing — Planned

Verification result auditing and quality assurance

Formalization, completed

SpecVerify: natural-language requirements to checkable assertions, published at IEEE RE 2025.

Decomposition, in progress

System-level properties decomposed into component obligations, driven by counterexample-guided inductive synthesis.

Auditing, planned

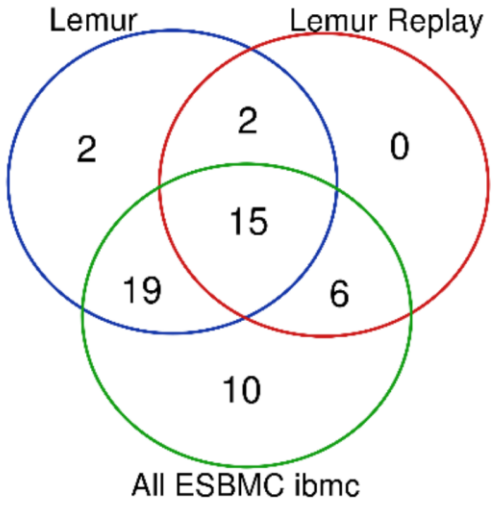
Property validation and coverage analysis: evidence that the properties checked were the ones that mattered.

Stage 3 is where an audit trail for AI-generated artefacts would be built.

ibmc and LEMUR: a portfolio beats either

MANCHESTER 1824
The University of Manchester

Comparison to LEMUR



LEMUR is similar as it also uses an LLM to suggest invariants.

We could not run LEMUR so replayed their invariants using our extended ESBMC

Using their invariants we verified 6 more programs

Using our invariants we verified 10 more programs

Potential lessons from both sides

Portfolio-style cooperation between ibmc and LEMUR!

Reproduced from the Coop 2026 deck.

Prompt engineering for invariant generation

MANCHESTER 1824
The University of Manchester

Prompt Engineering for Invariant Generation

```
<Examples with explanation>

Based on these examples provided above can you
generate  a C invariant for the following code,

<Labelled Input C Program>

Print an invariant for this loop that holds in the
form '__invariant(...);'. They should help prove the
assertions. You can utilise '&&' or '||' if required.
No explanation.  Your answer should be in the form
'__invariant(. . .);'
```

Chain of Thought (CoT) Prompt Engineering

Small number of different prompts that vary by their examples e.g. :

- Single Invariant
- Multiple Invariants

Full prompts (more explanation), constrained prompts (less explanation), and combined prompts

More explanation helps, but can distract, so a balance is best; further tuning may improve results

Reproduced from the Coop 2026 deck.

BACKUP D1

Four Turing Awards built the ground this talk stands on

Automated program reasoning: a sixty-year research programme.

1978

Robert W. Floyd

Founding automatic program verification as a subfield

How do you even state that a program is correct?

1980

C. A. R. Hoare

Fundamental contributions to the definition and design of programming languages

Hoare logic: preconditions, postconditions, invariants.

1996

Amir Pnueli

Introducing temporal logic into computing science; program and system verification

How do you state a property that must hold over time?

2007

Clarke, Emerson and Sifakis

Developing model checking into a highly effective verification technology

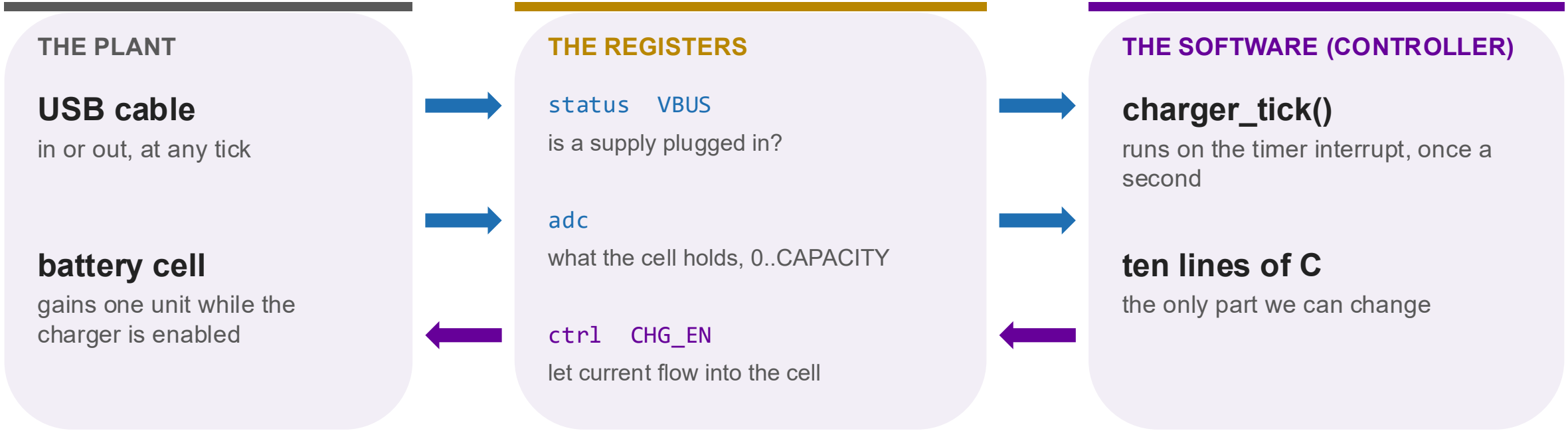
How do you check it automatically? ESBMC is a direct descendant.

Every award was for making correctness machine-checkable; that is still open, and generative AI makes it urgent.

A good field to join: the foundations are settled, the automation is not.

The program: ten lines of code, wired to a battery

A single-cell charger: the software reads two registers and writes one bit.



What the harness leaves open

The cable at every tick, the ADC reading, how long the device runs.

What stays concrete

The cell: enabled means one unit more, and never more than CAPACITY.

> The code is ten lines. The hardware it is wired to is the other half of the specification.

turing-awards-esbmc/charger.h and charger.c; the harness that plays the hardware is 1-floyd.c.

One program, four ways to say what "correct" means

The same battery charger, specified four ways, one per award, each checked by ESBMC.

1978 **Floyd**	State it at a point. `__ESBMC_assert(battery <= CAPACITY, "never overcharged");`	**Proved for every run**

1980
Hoare
	State it compositionally, and inductively.
	`__ESBMC_requires / __ESBMC_ensures`
	`__ESBMC_loop_invariant(battery <= CAPACITY);`
		Proved where BMC gives up

1996 **Pnueli**	State what must hold over time. `G( pressed → F (charge > min) )`	**No violating trace within the bound**
2007 **Clarke, Emerson & Sifakis**	Write nothing new; the machine decides. *every verdict above, reached automatically*	**A proof, or a counterexample**

Three languages for saying what correct means; one machine that checks all three.

References and sources

Cordeiro, L., Fischer, B., Marques-Silva, J.	SMT-Based Bounded Model Checking for Embedded ANSI-C Software. ASE 2009, pp. 137–148 (ASE 2023 Most Influential Paper); IEEE TSE 38(4):957–974, 2012.
Alshmrany, K., Aldughaim, M., Bhayat, A., Cordeiro, L.	FuSeBMC v4: Improving Code Coverage with Smart Seeds via BMC, Fuzzing and Static Analysis. Formal Aspects of Computing 36(2), 2024, doi:10.1145/3665337.
Pirzada, M. A. A., Reger, G., Bhayat, A., Cordeiro, L.	LLM-Generated Invariants for Bounded Model Checking Without Loop Unrolling. ASE 2024, pp. 1395–1407. DOI 10.1145/3691620.3695512.
Wang, W., Farrell, M., Cordeiro, L., Zhao, L.	Supporting Software Formal Verification with Large Language Models: An Experimental Study. IEEE RE 2025, pp. 423–431. DOI 10.1109/RE63999.2025.00049.
Tihanyi, N., Bisztray, T., Ferrag, M. A., Jain, R., Cordeiro, L.	How secure is AI-generated code: a large-scale comparison of large language models. Empirical Software Engineering 30(2):47, 2025. DOI 10.1007/s10664-024-10590-1.
Mahmud, Sikder, Ji, Zhang, Lei	The Illusion of Safety: Multi-Tier Verification of AI vs. Human C++ Code. arXiv:2607.00107, 2026 (preprint).
Bessey, A., et al.	A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. CACM 53(2):66–75, 2010. DOI 10.1145/1646353.1646374.
Sadowski, C., et al.	Tricorder: Building a Program Analysis Ecosystem. ICSE 2015, pp. 598–608. DOI 10.1109/ICSE.2015.76.

Non-bibliographic sources

ESBMC: github.com/esbmc/esbmc · Harnesses: github.com/lucasccordeiro · AWS: bulletin 2026-080-AWS, GHSA-wxx3-prfc-69xx, GHSA-mxm9-xpf9-x66x · NVIDIA/OpenSMA issue #1 · vllm-project/vllm #43496 · Competitions: sv-comp.sosy-lab.org/2026 · test-comp.sosy-lab.org/2026 · Rust: rustfoundation.org, 7 Aug 2025 · AI-code shares: blog.google; anthropic.com/institute; TechCrunch; TechSpot.